

Transformer Architectures and Variants

A Mathematical and Architectural Treatment

Part 5

Efficient, Sparse, Long-Context, Retrieval-Based, and Modern Transformer Architectures

Contents

1	The Need for Efficient Transformers	1
2	Longformer	3
2.1	Motivation	3
2.2	Input Representation	3
2.3	Sliding-Window Attention	3
2.4	Global Attention	4
2.5	Complexity	4
2.6	Why Longformer Was Introduced	4
2.7	Applications	5
3	BigBird	6
3.1	Motivation	6
3.2	Three Attention Patterns	6
3.3	Sparse Attention	6
3.4	Why Random Connections Matter	7
3.5	Complexity	7
3.6	Applications	7
4	Reformer	8
4.1	Motivation	8
4.2	Locality-Sensitive Hashing Attention	8
4.3	Attention After Hashing	8
4.4	Reversible Transformer Layers	9
4.5	Architectural Change	9
5	Linformer	10
5.1	Motivation	10
5.2	Projection of Keys and Values	10
5.3	Attention	11
5.4	Complexity	11
6	Performer	12
6.1	Motivation	12
6.2	Kernel Representation	12
6.3	Rewriting Attention	12

6.4 Complexity	13
7 FlashAttention	14
7.1 Motivation	14
7.2 Standard Attention	14
7.3 Tiling	14
7.4 Online Softmax	15
7.5 Architectural Significance	15
8 Multi-Query Attention	17
8.1 Motivation	17
8.2 Standard Multi-Head Attention	17
8.3 Multi-Query Modification	18
8.4 Why This Helps	18
9 Grouped-Query Attention	19
9.1 Motivation	19
9.2 Groups	19
9.3 Relationship Between Attention Variants	20
10 Rotary Position Embeddings	21
10.1 Motivation	21
10.2 Two-Dimensional Rotation	21
10.3 Application to Queries and Keys	21
10.4 Why RoPE Is Important	22
11 ALiBi	23
11.1 Motivation	23
11.2 Attention Scores	23
11.3 Interpretation	23
12 Retrieval-Augmented Transformers	24
12.1 Motivation	24
12.2 Query Representation	24
12.3 Retrieved Context	24
12.4 Why Retrieval Changes the Architecture	25
13 RETRO	26
13.1 Motivation	26
13.2 Chunking	26
13.3 Retrieval Cross-Attention	26
14 Mixture-of-Experts Transformers	28
14.1 Motivation	28
14.2 Experts	28
14.3 Router	28

14.4 Sparse Activation	29
14.5 Where MoE Appears	29
15 Switch Transformer	30
15.1 Motivation	30
15.2 Router	30
15.3 Difference from Dense Transformer	30
16 Mixture-of-Experts Routing and Load Balancing	32
16.1 The Routing Problem	32
16.2 Auxiliary Routing Objective	32
17 Encoder-Decoder Transformers Revisited	34
17.1 Encoder-Only	34
17.2 Decoder-Only	35
17.3 Encoder-Decoder	35
18 BERT	36
18.1 Motivation	36
18.2 Input	36
18.3 Bidirectional Self-Attention	36
18.4 Masked Language Modeling	37
19 GPT-Style Decoder Transformers	38
19.1 Motivation	38
19.2 Causal Mask	38
19.3 Next-Token Prediction	38
20 Modern Transformer Design: Combining Variants	40
21 SwiGLU Feed-Forward Networks	41
21.1 Motivation	41
21.2 Swish	41
21.3 SwiGLU	41
22 RMSNorm	42
22.1 Motivation	42
22.2 RMS	42
23 Summary of Part 5	43
23.1 Attention Pattern	43
23.2 Attention Approximation	43
23.3 Attention Implementation	43
23.4 Key-Value Sharing	44
23.5 Position Representation	44
23.6 External Memory	44

23.7 Parameter Sparsity	44
23.8 Overall View	45

Chapter 1

The Need for Efficient Transformers

The standard self-attention mechanism receives a sequence

$$X \in \mathbb{R}^{N \times d}. \quad (1.1)$$

The query, key, and value matrices are

$$Q = XW^Q, \quad (1.2)$$

$$K = XW^K, \quad (1.3)$$

$$V = XW^V. \quad (1.4)$$

The attention matrix is

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right). \quad (1.5)$$

Since

$$QK^T \in \mathbb{R}^{N \times N}, \quad (1.6)$$

the attention computation has quadratic dependence on the sequence length:

$$O(N^2d). \quad (1.7)$$

For language sequences, time series, images, and scientific datasets, N can become very large.

This motivates several families of Transformer variants:

sparse attention,
local attention,
linear attention,
low-rank attention,
memory-based attention,
recurrent/state-space alternatives,
retrieval-augmented attention.

(1.8)

The following architectures modify the Transformer primarily to address computational complexity or long-context processing.

Chapter 2

Longformer

2.1 Motivation

Longformer was designed for documents containing thousands or tens of thousands of tokens.

Paper:

Longformer: The Long-Document Transformer

The fundamental modification is replacing unrestricted global attention with a combination of local and selected global attention.

2.2 Input Representation

Let

$$X = [x_1, x_2, \dots, x_N] \tag{2.1}$$

be the embedded token sequence.

Thus

$$X \in \mathbb{R}^{N \times d}. \tag{2.2}$$

Unlike the original Transformer, token i does not necessarily attend to every token.

2.3 Sliding-Window Attention

For token i , define a local window

$$\mathcal{W}_i = \{j : |i - j| \leq w\}. \tag{2.3}$$

The query for token i is

$$q_i = x_i W^Q. \tag{2.4}$$

Keys and values are restricted to the local window:

$$K_i = \{x_j W^K : j \in \mathcal{W}_i\}, \quad (2.5)$$

$$V_i = \{x_j W^V : j \in \mathcal{W}_i\}. \quad (2.6)$$

The attention output becomes

$$z_i = \text{softmax} \left(\frac{q_i K_i^T}{\sqrt{d_k}} \right) V_i. \quad (2.7)$$

The number of attended positions is approximately

$$2w + 1 \quad (2.8)$$

rather than N .

2.4 Global Attention

Some positions are designated as global attention positions.

For a global token g , attention may be computed against the whole sequence:

$$z_g = \text{softmax} \left(\frac{q_g K^T}{\sqrt{d_k}} \right) V. \quad (2.9)$$

Therefore Longformer combines

$$\boxed{\text{local attention} + \text{global attention}.} \quad (2.10)$$

2.5 Complexity

With a fixed window size w , local attention has approximate complexity

$$O(Nwd). \quad (2.11)$$

If w is fixed,

$$O(Nwd) \sim O(N). \quad (2.12)$$

This avoids the $O(N^2d)$ complexity of global attention.

2.6 Why Longformer Was Introduced

The original Transformer assumes that every token can attend to every other token.

Longformer instead assumes that most interactions can be handled locally, while a relatively small number of positions require global communication.

This is particularly suitable for structured long documents.

2.7 Applications

Longformer is useful for

- long-document classification,
- document question answering,
- legal documents,
- scientific documents,
- long-context NLP.

Repository:

[Longformer implementation](#)

Chapter 3

BigBird

3.1 Motivation

BigBird also addresses the quadratic complexity of Transformer attention for long sequences.

Paper:

Big Bird: Transformers for Longer Sequences

Instead of one attention pattern, BigBird combines several types of sparse connections.

3.2 Three Attention Patterns

For a token i , BigBird can establish connections through:

1. local attention,
2. random attention,
3. global attention.

Let

$$\mathcal{N}_i \tag{3.1}$$

be the set of positions attended by token i .

Then

$$\mathcal{N}_i = \mathcal{N}_i^{\text{local}} \cup \mathcal{N}_i^{\text{random}} \cup \mathcal{N}_i^{\text{global}}. \tag{3.2}$$

3.3 Sparse Attention

For each query,

$$q_i = x_i W^Q. \tag{3.3}$$

Only keys corresponding to

$$j \in \mathcal{N}_i \quad (3.4)$$

are used.

Therefore,

$$z_i = \text{softmax} \left(\frac{q_i K_{\mathcal{N}_i}^T}{\sqrt{d_k}} \right) V_{\mathcal{N}_i}. \quad (3.5)$$

3.4 Why Random Connections Matter

Local attention provides short-range communication.

However, if only local connections existed, information would need many layers to propagate across the sequence.

Random connections create long-range shortcuts.

A simplified connectivity structure is therefore

$$\text{local} + \text{random} + \text{global}. \quad (3.6)$$

Repeated sparse layers allow information to propagate over a large sequence without constructing the full $N \times N$ attention matrix.

3.5 Complexity

If each token attends to a fixed number of positions s , then

$$O(Nsd). \quad (3.7)$$

For fixed s ,

$$O(N). \quad (3.8)$$

3.6 Applications

BigBird is particularly useful for

- long-document processing,
- question answering,
- summarization,
- genomic sequences,
- long-context representation learning.

Repository:

[BigBird implementation](#)

Chapter 4

Reformer

4.1 Motivation

Reformer attempts to reduce the memory and computational cost of Transformer attention.

Paper:

Reformer: The Efficient Transformer

Two important modifications are:

locality-sensitive hashing attention + reversible layers.

(4.1)

4.2 Locality-Sensitive Hashing Attention

The key observation is that similar queries should preferentially interact with similar keys.

Instead of computing every pairwise similarity

$$q_i^T k_j, \tag{4.2}$$

queries and keys are grouped using a hashing function.

Let

$$h(q_i) \tag{4.3}$$

denote the hash bucket assigned to query q_i .

Tokens with similar representations are encouraged to occupy the same bucket.

Attention is then computed within candidate buckets rather than over the entire sequence.

4.3 Attention After Hashing

Suppose token i belongs to bucket b .

Let

$$\mathcal{B}_b \tag{4.4}$$

be the tokens assigned to that bucket.

Then

$$z_i = \text{softmax} \left(\frac{q_i K_{\mathcal{B}_b}^T}{\sqrt{d_k}} \right) V_{\mathcal{B}_b}. \quad (4.5)$$

Thus the candidate set is much smaller than N .

4.4 Reversible Transformer Layers

A conventional Transformer stores intermediate activations for backpropagation.

Reformer uses reversible layers.

Split the input into two parts:

$$x = (x_1, x_2). \quad (4.6)$$

The forward equations can be written as

$$y_1 = x_1 + F(x_2), \quad (4.7)$$

$$y_2 = x_2 + G(y_1). \quad (4.8)$$

The original variables can be reconstructed:

$$x_2 = y_2 - G(y_1), \quad (4.9)$$

and

$$x_1 = y_1 - F(x_2). \quad (4.10)$$

Therefore intermediate activations do not all need to be stored.

4.5 Architectural Change

The original Transformer has

$$\text{dense attention} + \text{stored activations}. \quad (4.11)$$

Reformer replaces these with

$$\text{hashed sparse attention} + \text{reversible computation}. \quad (4.12)$$

Repository:

[Trax implementation](#)

Chapter 5

Linformer

5.1 Motivation

Linformer observes that the attention matrix often has an approximately low-rank structure.

Paper:

Linformer: Self-Attention with Linear Complexity

The original attention is

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right). \quad (5.1)$$

The matrix

$$QK^T \quad (5.2)$$

has dimensions

$$N \times N. \quad (5.3)$$

Linformer reduces the effective sequence dimension of keys and values.

5.2 Projection of Keys and Values

Let

$$E \in \mathbb{R}^{N \times k} \quad (5.4)$$

and

$$F \in \mathbb{R}^{N \times k}, \quad (5.5)$$

where

$$k \ll N. \quad (5.6)$$

Project keys:

$$K' = E^T K. \quad (5.7)$$

Project values:

$$V' = F^T V. \quad (5.8)$$

Therefore,

$$K' \in \mathbb{R}^{k \times d_k}, \quad (5.9)$$

and

$$V' \in \mathbb{R}^{k \times d_v}. \quad (5.10)$$

5.3 Attention

The new attention operation is

$$Z = \text{softmax} \left(\frac{Q(K')^T}{\sqrt{d_k}} \right) V'. \quad (5.11)$$

The attention matrix is now

$$N \times k \quad (5.12)$$

instead of

$$N \times N. \quad (5.13)$$

5.4 Complexity

The main attention interaction becomes approximately

$$O(Nkd). \quad (5.14)$$

If k is independent of N ,

$$O(N). \quad (5.15)$$

This is the central mathematical modification introduced by Linformer.

Repository:

[Linformer PyTorch implementation](#)

Chapter 6

Performer

6.1 Motivation

Performer approximates softmax attention using kernel methods.

Paper:

Rethinking Attention with Performers

The standard attention operation contains

$$\exp(q^T k). \tag{6.1}$$

Computing all pairwise values creates the quadratic bottleneck.

Performer replaces explicit pairwise computation with a feature map approximation.

6.2 Kernel Representation

Suppose

$$K(q, k) = \exp(q^T k). \tag{6.2}$$

Assume a feature map ϕ such that

$$K(q, k) \approx \phi(q)^T \phi(k). \tag{6.3}$$

Then

$$\exp(q^T k) \approx \phi(q)^T \phi(k). \tag{6.4}$$

6.3 Rewriting Attention

Standard attention for a query q_i is

$$z_i = \frac{\sum_j \exp(q_i^T k_j) v_j}{\sum_j \exp(q_i^T k_j)}. \tag{6.5}$$

Using the feature map,

$$z_i \approx \frac{\sum_j \phi(q_i)^T \phi(k_j) v_j}{\sum_j \phi(q_i)^T \phi(k_j)}. \quad (6.6)$$

Factor out $\phi(q_i)^T$:

$$z_i \approx \frac{\phi(q_i)^T \left(\sum_j \phi(k_j) v_j^T \right)}{\phi(q_i)^T \left(\sum_j \phi(k_j) \right)}. \quad (6.7)$$

The sums over keys and values can be computed before processing each query. This avoids explicitly constructing the complete attention matrix.

6.4 Complexity

The resulting computation can scale approximately linearly with sequence length:

$$O(Nd^2) \quad (6.8)$$

under appropriate feature-map dimensions, rather than

$$O(N^2d). \quad (6.9)$$

Repository:

[Performer implementation](#)

Chapter 7

FlashAttention

7.1 Motivation

FlashAttention does not fundamentally change the mathematical definition of attention.

Instead, it changes how attention is computed on hardware.

Paper:

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

This distinction is important:

FlashAttention is exact attention, not an approximate attention formula.

 (7.1)

7.2 Standard Attention

The standard operation is

$$S = \frac{QK^T}{\sqrt{d_k}}, \quad (7.2)$$

followed by

$$P = \text{softmax}(S), \quad (7.3)$$

and

$$O = PV. \quad (7.4)$$

The intermediate matrix S has size

$$N \times N. \quad (7.5)$$

Writing this matrix to and reading it from high-bandwidth memory can be expensive.

7.3 Tiling

FlashAttention partitions Q , K , and V into blocks.

Let

$$Q_i \tag{7.6}$$

be a query block and

$$K_j, V_j \tag{7.7}$$

be key-value blocks.

The local score block is

$$S_{ij} = \frac{Q_i K_j^T}{\sqrt{d_k}}. \tag{7.8}$$

Rather than materializing the complete matrix

$$S, \tag{7.9}$$

the algorithm processes blocks sequentially.

7.4 Online Softmax

The softmax normalization is maintained incrementally using running statistics.

For a row, maintain

$$m \tag{7.10}$$

for the running maximum and

$$\ell \tag{7.11}$$

for the running normalization term.

When a new block is encountered, the statistics are updated so that the final result is mathematically equivalent to the ordinary softmax.

Thus,

$$\text{FlashAttention}(Q, K, V) = \text{Attention}(Q, K, V). \tag{7.12}$$

The difference lies in memory access and computation scheduling.

7.5 Architectural Significance

FlashAttention therefore belongs to a different category from Longformer or Linformer.

Longformer changes

$$\text{which tokens attend to which tokens.} \tag{7.13}$$

Linformer changes

the representation of K, V . (7.14)

FlashAttention keeps

the same mathematical attention (7.15)

while changing its implementation.

Repository:

[FlashAttention implementation](#)

Chapter 8

Multi-Query Attention

8.1 Motivation

In autoregressive generation, the model repeatedly uses previously computed keys and values.

Standard multi-head attention maintains separate key and value projections for every head.

For h heads,

$$K_1, \dots, K_h \tag{8.1}$$

and

$$V_1, \dots, V_h \tag{8.2}$$

must be stored.

This increases the size of the key-value cache.

Multi-Query Attention shares keys and values across heads.

Paper:

Fast Transformer Decoding: One Write-Head is All You Need

8.2 Standard Multi-Head Attention

For head i ,

$$Q_i = XW_i^Q, \tag{8.3}$$

$$K_i = XW_i^K, \tag{8.4}$$

$$V_i = XW_i^V. \tag{8.5}$$

Then

$$Z_i = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i. \tag{8.6}$$

8.3 Multi-Query Modification

Each head still has a separate query:

$$Q_i = XW_i^Q. \quad (8.7)$$

However, all heads share

$$K = XW^K \quad (8.8)$$

and

$$V = XW^V. \quad (8.9)$$

Therefore

$$Z_i = \text{softmax} \left(\frac{Q_i K^T}{\sqrt{d_k}} \right) V. \quad (8.10)$$

The outputs are then concatenated:

$$Z = \text{Concat}(Z_1, \dots, Z_h). \quad (8.11)$$

8.4 Why This Helps

During autoregressive decoding, the model must store past keys and values.

Sharing them across heads reduces the amount of key-value memory that must be maintained.

Thus the primary advantage is inference efficiency.

Chapter 9

Grouped-Query Attention

9.1 Motivation

Multi-Query Attention shares one key-value head among all query heads.

This can improve efficiency but may reduce representational capacity.

Grouped-Query Attention provides an intermediate design.

Paper:

GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints

9.2 Groups

Suppose there are

$$h_q \tag{9.1}$$

query heads and

$$h_{kv} \tag{9.2}$$

key-value heads, where

$$h_{kv} \leq h_q. \tag{9.3}$$

Query heads are divided into groups.

Each group shares a key-value head.

For group g ,

$$K_g = XW_g^K, \tag{9.4}$$

$$V_g = XW_g^V. \tag{9.5}$$

A query head i assigned to group $g(i)$ uses

$$Z_i = \text{softmax} \left(\frac{Q_i K_{g(i)}^T}{\sqrt{d_k}} \right) V_{g(i)}. \quad (9.6)$$

9.3 Relationship Between Attention Variants

If

$$h_{kv} = h_q, \quad (9.7)$$

then every query head has its own key-value head:

$$\text{GQA} \rightarrow \text{MHA}. \quad (9.8)$$

If

$$h_{kv} = 1, \quad (9.9)$$

then

$$\text{GQA} \rightarrow \text{MQA}. \quad (9.10)$$

Thus

$$\boxed{\text{MHA} \longleftrightarrow \text{GQA} \longleftrightarrow \text{MQA}} \quad (9.11)$$

forms a continuum between maximum key-value independence and maximum key-value sharing.

Chapter 10

Rotary Position Embeddings

10.1 Motivation

The Transformer needs positional information.

Traditional Transformer architectures add a positional embedding directly to token representations.

Rotary Position Embedding, or RoPE, instead incorporates position into the query and key vectors through rotations.

Paper:

RoFormer: Enhanced Transformer with Rotary Position Embedding

10.2 Two-Dimensional Rotation

Consider a pair of coordinates

$$(x_1, x_2). \tag{10.1}$$

A rotation by angle θ is

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}. \tag{10.2}$$

Thus

$$R(\theta) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_1 \cos \theta - x_2 \sin \theta \\ x_1 \sin \theta + x_2 \cos \theta \end{pmatrix}. \tag{10.3}$$

10.3 Application to Queries and Keys

For token position m , rotate the query:

$$q'_m = R_m q_m. \tag{10.4}$$

Similarly,

$$k'_n = R_n k_n. \quad (10.5)$$

The attention score becomes

$$(q'_m)^T k'_n = (R_m q_m)^T (R_n k_n). \quad (10.6)$$

Using

$$R_m^T R_n = R_{n-m}, \quad (10.7)$$

the relative displacement between positions appears naturally.

Thus the attention score can encode relative positional information through the difference

$$n - m. \quad (10.8)$$

10.4 Why RoPE Is Important

The original sinusoidal encoding adds position to the token representation.

RoPE modifies the query and key geometry themselves.

This makes relative position naturally influence their inner product.

RoPE is therefore particularly important in modern autoregressive language models.

Chapter 11

ALiBi

11.1 Motivation

Another approach to position is to directly modify the attention scores using a distance-dependent bias.

Paper:

Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation

11.2 Attention Scores

The ordinary attention score is

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}}. \quad (11.1)$$

ALiBi adds a position-dependent bias:

$$s'_{ij} = s_{ij} - m|i - j|, \quad (11.2)$$

where m is a slope.

The attention becomes

$$A_{ij} = \text{softmax}_j (s'_{ij}). \quad (11.3)$$

11.3 Interpretation

The term

$$-m|i - j| \quad (11.4)$$

penalizes distant tokens.

Therefore the model has an explicit preference for nearby positions while still permitting long-range attention.

Chapter 12

Retrieval-Augmented Transformers

12.1 Motivation

A conventional Transformer stores knowledge in its parameters.

A retrieval-augmented Transformer supplements parametric knowledge with external information.

The conceptual architecture is

$$\boxed{\text{query} \rightarrow \text{retrieval} \rightarrow \text{relevant documents} \rightarrow \text{Transformer}.} \quad (12.1)$$

12.2 Query Representation

Given an input

$$x, \quad (12.2)$$

an encoder produces a query representation

$$q = f_{\theta}(x). \quad (12.3)$$

A database contains document embeddings

$$d_1, \dots, d_M. \quad (12.4)$$

Similarity can be measured using

$$s(q, d_i) = q^T d_i. \quad (12.5)$$

The top- k documents are selected:

$$D_k = \text{TopK}(s(q, d_i)). \quad (12.6)$$

12.3 Retrieved Context

The selected documents are

$$D_k = [d_{i_1}, \dots, d_{i_k}]. \quad (12.7)$$

They can then be incorporated into the Transformer input. Conceptually,

$$X' = [X; D_k]. \quad (12.8)$$

The Transformer then computes

$$Y = f_\theta(X'). \quad (12.9)$$

12.4 Why Retrieval Changes the Architecture

The original Transformer computes

$$Y = f_\theta(X). \quad (12.10)$$

A retrieval-augmented system computes

$$Y = f_\theta(X, \text{Retrieve}(X)). \quad (12.11)$$

Thus the model can access information that does not need to be encoded entirely in its fixed parameters.

Chapter 13

RETRO

13.1 Motivation

RETRO integrates retrieval directly into a language model.

Paper:

Improving Language Models by Retrieving From Trillions of Tokens

The central idea is that a language model can retrieve similar text sequences from a large external database.

13.2 Chunking

The input sequence is divided into chunks:

$$X = [x_1, \dots, x_N] \rightarrow C_1, \dots, C_m. \quad (13.1)$$

Each chunk is represented by an embedding.

A nearest-neighbor search retrieves related chunks:

$$R_i = \text{Retrieve}(C_i). \quad (13.2)$$

13.3 Retrieval Cross-Attention

The Transformer representation supplies queries:

$$Q = HW^Q. \quad (13.3)$$

Retrieved representations supply keys and values:

$$K_R = RW^K, \quad (13.4)$$

$$V_R = RW^V. \quad (13.5)$$

Retrieval attention becomes

$$Z_R = \text{softmax} \left(\frac{QK_R^T}{\sqrt{d_k}} \right) V_R. \quad (13.6)$$

The retrieved information is then incorporated into the Transformer representation. Thus RETRO adds an external information pathway to the Transformer.

Chapter 14

Mixture-of-Experts Transformers

14.1 Motivation

Increasing the number of model parameters can improve model capacity.

However, activating every parameter for every token is expensive.

Mixture-of-Experts, or MoE, addresses this by activating only a subset of expert networks for each token.

14.2 Experts

Suppose there are M expert networks:

$$E_1, \dots, E_M. \quad (14.1)$$

Each expert can be a feed-forward network:

$$E_i(x) = W_{i,2}\sigma(W_{i,1}x + b_{i,1}) + b_{i,2}. \quad (14.2)$$

14.3 Router

A router receives token representation x :

$$r(x) = \text{softmax}(W_r x). \quad (14.3)$$

The router produces

$$r(x) = [r_1(x), \dots, r_M(x)]. \quad (14.4)$$

The model selects the top- k experts.

Let

$$\mathcal{T}(x) \quad (14.5)$$

be the selected expert set.

The MoE output is

$$y = \sum_{i \in \mathcal{T}(x)} r_i(x) E_i(x). \quad (14.6)$$

14.4 Sparse Activation

If there are M experts but only k are activated,

$$k \ll M, \quad (14.7)$$

then the total parameter count can be very large while the computation for one token remains associated mainly with the selected experts.

This produces the important distinction:

$$\boxed{\text{parameter count} \neq \text{parameters activated per token.}} \quad (14.8)$$

14.5 Where MoE Appears

MoE layers are often inserted in place of the conventional Transformer feed-forward network:

$$X \rightarrow \text{Attention}(X) \rightarrow \text{MoE}(X). \quad (14.9)$$

Thus the attention mechanism itself may remain unchanged.

The major modification is the FFN sublayer.

Chapter 15

Switch Transformer

15.1 Motivation

Switch Transformer simplifies MoE routing by sending each token to a single expert.

Paper:

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

15.2 Router

For token representation x ,

$$p = \text{softmax}(W_r x). \quad (15.1)$$

The selected expert is

$$i^* = \arg \max_i p_i. \quad (15.2)$$

Only that expert processes the token:

$$y = E_{i^*}(x). \quad (15.3)$$

A routing probability can also scale the output:

$$y = p_{i^*} E_{i^*}(x). \quad (15.4)$$

15.3 Difference from Dense Transformer

Dense Transformer:

$$y = E(x). \quad (15.5)$$

Switch Transformer:

$$y = E_{i^*(x)}(x). \quad (15.6)$$

Therefore different tokens can be processed by different expert networks.

Chapter 16

Mixture-of-Experts Routing and Load Balancing

16.1 The Routing Problem

If the router sends almost every token to the same expert, the other experts receive little training.

Suppose expert i receives

$$n_i \tag{16.1}$$

tokens.

Ideally,

$$n_i \approx \frac{N}{M}. \tag{16.2}$$

A load-balancing objective therefore encourages approximately uniform routing.

16.2 Auxiliary Routing Objective

Let

$$f_i \tag{16.3}$$

represent the fraction of tokens assigned to expert i .

Let

$$p_i \tag{16.4}$$

represent the average router probability assigned to expert i .

A representative balancing objective has the form

$$\mathcal{L}_{\text{balance}} \propto M \sum_{i=1}^M f_i p_i. \tag{16.5}$$

The exact formulation varies between MoE architectures, but the purpose is the same:

prevent expert collapse.

(16.6)

Chapter 17

Encoder-Decoder Transformers Revisited

The original Transformer contains two separate components:

$$\text{Encoder} + \text{Decoder}. \quad (17.1)$$

The encoder produces

$$H = \text{Encoder}(X). \quad (17.2)$$

The decoder performs masked self-attention over the generated sequence and cross-attention over the encoder output.

For decoder state Y ,

$$Q = YW^Q, \quad (17.3)$$

while

$$K = HW^K, \quad (17.4)$$

$$V = HW^V. \quad (17.5)$$

Cross-attention is

$$Z = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V. \quad (17.6)$$

Modern Transformer models can therefore be grouped according to which components they retain.

17.1 Encoder-Only

Encoder-only models compute

$$H = \text{Encoder}(X). \quad (17.7)$$

They are primarily suited to representation learning and understanding.
Examples include BERT-like architectures.

17.2 Decoder-Only

Decoder-only models use causal self-attention:

$$A_{ij} = 0 \quad \text{for} \quad j > i. \quad (17.8)$$

Thus token i can only access

$$x_1, \dots, x_i. \quad (17.9)$$

This produces autoregressive generation:

$$p(x_1, \dots, x_N) = \prod_{i=1}^N p(x_i \mid x_{<i}). \quad (17.10)$$

17.3 Encoder-Decoder

Encoder-decoder models retain both components:

$$X \rightarrow H \rightarrow Y. \quad (17.11)$$

They are particularly natural for conditional generation.

Examples include translation, summarization, and many sequence-to-sequence scientific tasks.

Chapter 18

BERT

18.1 Motivation

BERT adapts the Transformer encoder to bidirectional language representation learning.

Paper:

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

18.2 Input

The input consists of tokens

$$X = [x_1, \dots, x_N]. \quad (18.1)$$

Each token representation is constructed from token and positional information.

Conceptually,

$$H^{(0)} = E_{\text{token}} + E_{\text{position}} + E_{\text{segment}}. \quad (18.2)$$

18.3 Bidirectional Self-Attention

Unlike an autoregressive decoder, BERT permits token i to attend to tokens on both sides.

Therefore

$$A_{ij} \neq 0 \quad (18.3)$$

for both

$$j < i \quad (18.4)$$

and

$$j > i. \quad (18.5)$$

The standard attention equation remains

$$Z = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V. \quad (18.6)$$

The major modification is therefore the training objective and use of the encoder rather than a new attention formula.

18.4 Masked Language Modeling

A subset of tokens

$$\mathcal{M} \quad (18.7)$$

is masked.

For $i \in \mathcal{M}$, the model predicts

$$p(x_i \mid X_{\setminus \mathcal{M}}). \quad (18.8)$$

The loss is

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log p(x_i \mid X_{\setminus \mathcal{M}}). \quad (18.9)$$

Chapter 19

GPT-Style Decoder Transformers

19.1 Motivation

Decoder-only Transformers provide a direct framework for autoregressive language generation.

For a sequence

$$X = (x_1, \dots, x_N), \quad (19.1)$$

the model estimates

$$p(x_i \mid x_1, \dots, x_{i-1}). \quad (19.2)$$

The complete probability is

$$p(X) = \prod_{i=1}^N p(x_i \mid x_{<i}). \quad (19.3)$$

19.2 Causal Mask

The attention score matrix is masked:

$$S_{ij} = \begin{cases} \frac{q_i^T k_j}{\sqrt{d_k}}, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (19.4)$$

Then

$$A = \text{softmax}(S). \quad (19.5)$$

Consequently,

$$A_{ij} = 0 \quad j > i. \quad (19.6)$$

19.3 Next-Token Prediction

The final hidden representation at position i is

$$h_i. \tag{19.7}$$

The vocabulary logits are

$$z_i = Wh_i + b. \tag{19.8}$$

The next-token probability is

$$p(x_{i+1} \mid x_{\leq i}) = \text{softmax}(z_i). \tag{19.9}$$

The model is therefore trained through

$$\mathcal{L} = - \sum_i \log p(x_{i+1} \mid x_{\leq i}). \tag{19.10}$$

Chapter 20

Modern Transformer Design: Combining Variants

Modern architectures generally do not correspond to only one variant.

A contemporary Transformer may simultaneously use:

decoder-only architecture,
RoPE positional encoding,
GQA,
RMSNorm,
SwiGLU FFN,
FlashAttention,
Mixture-of-Experts,
long-context mechanisms.

(20.1)

These modifications operate at different levels.

For example,

RoPE

(20.2)

changes positional representation.

GQA

(20.3)

changes the relationship between query and key-value heads.

FlashAttention

(20.4)

changes the implementation of attention.

MoE

(20.5)

changes the feed-forward computation.

Therefore these variants are not necessarily mutually exclusive.

Chapter 21

SwiGLU Feed-Forward Networks

21.1 Motivation

The standard Transformer feed-forward network is

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2. \quad (21.1)$$

Modern Transformer architectures frequently replace this with gated activation functions such as SwiGLU.

21.2 Swish

The Swish activation is

$$\text{Swish}(x) = x \sigma(x), \quad (21.2)$$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (21.3)$$

21.3 SwiGLU

SwiGLU can be written as

$$\text{SwiGLU}(x) = \text{Swish}(xW) \odot (xV), \quad (21.4)$$

where $\odot$ denotes element-wise multiplication.

An output projection is then applied:

$$y = \text{SwiGLU}(x)W_o. \quad (21.5)$$

The gating mechanism allows one learned transformation to modulate another.

This is a modification of the Transformer FFN rather than the attention mechanism.

Chapter 22

RMSNorm

22.1 Motivation

The original Transformer uses Layer Normalization.

Given a vector

$$x \in \mathbb{R}^d, \tag{22.1}$$

LayerNorm computes its mean and variance.

RMSNorm removes the explicit mean-centering operation.

22.2 RMS

Define

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}. \tag{22.2}$$

Then

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\text{RMS}(x)}. \tag{22.3}$$

Thus normalization depends on the root-mean-square magnitude.

This is computationally simpler than full LayerNorm and is used in many modern Transformer implementations.

Chapter 23

Summary of Part 5

The architectures covered in this part modify Transformers at several distinct levels.

23.1 Attention Pattern

Longformer:

$$\text{global attention} \rightarrow \text{local} + \text{global attention.} \quad (23.1)$$

BigBird:

$$\text{global attention} \rightarrow \text{local} + \text{random} + \text{global attention.} \quad (23.2)$$

Reformer:

$$\text{dense attention} \rightarrow \text{LSH-based sparse attention.} \quad (23.3)$$

23.2 Attention Approximation

Linformer:

$$K, V \rightarrow E^T K, F^T V. \quad (23.4)$$

Performer:

$$\exp(q^T k) \rightarrow \phi(q)^T \phi(k). \quad (23.5)$$

23.3 Attention Implementation

FlashAttention keeps

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (23.6)$$

mathematically intact while computing it using an IO-efficient tiled algorithm.

23.4 Key-Value Sharing

MHA:

$$h_q = h_{kv}. \quad (23.7)$$

GQA:

$$1 < h_{kv} < h_q. \quad (23.8)$$

MQA:

$$h_{kv} = 1. \quad (23.9)$$

23.5 Position Representation

RoPE modifies query and key vectors:

$$q_i \rightarrow R_i q_i, \quad k_i \rightarrow R_i k_i. \quad (23.10)$$

ALiBi modifies attention logits:

$$s_{ij} \rightarrow s_{ij} - m|i - j|. \quad (23.11)$$

23.6 External Memory

Retrieval-based Transformers modify

$$f_\theta(X) \quad (23.12)$$

into

$$f_\theta(X, \text{Retrieve}(X)). \quad (23.13)$$

23.7 Parameter Sparsity

MoE changes the dense FFN

$$E(x) \quad (23.14)$$

into a routed collection

$$\sum_{i \in \mathcal{T}(x)} r_i(x) E_i(x). \quad (23.15)$$

Switch Transformer takes this further by selecting a single expert.

23.8 Overall View

The Transformer architecture should therefore not be viewed as a single fixed model.

Its major components can be independently modified:

$$\boxed{\text{Transformer} = \text{Representation} + \text{Attention} + \text{Position} + \text{FFN} + \text{Normalization} + \text{Routing} + \text{Memory}.}$$
(23.16)

Different Transformer variants arise by modifying one or more of these components.

For example:

$$\begin{aligned} &\text{ViT} : \text{image representation,} \\ &\text{Swin} : \text{attention pattern} + \text{hierarchy,} \\ &\text{Longformer} : \text{sparse attention,} \\ &\text{Linformer} : \text{low-rank attention,} \\ &\text{Performer} : \text{kernelized attention,} \\ &\text{FlashAttention} : \text{attention implementation,} \\ &\text{RoPE} : \text{positional representation,} \\ &\text{GQA} : \text{key-value sharing,} \\ &\text{MoE} : \text{sparse FFN routing,} \\ &\text{RETRO} : \text{external retrieval.} \end{aligned}$$
(23.17)

The result is a large design space rather than a single architecture.